



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 5.2

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción.....	3
1.1. Contexto.....	3
1.2. Objetivos del Sprint 5.2.....	3
1.3. Justificación.....	3
2. Metodología.....	4
2.1. Respuesta a HU-7: Análisis en Tiempo Real con Spark Structured Streaming.....	4
2.2. Configuración del Job de Spark Structured Streaming.....	4
2.3. Cálculo del VWAP por Ventanas de 5 Minutos.....	5
2.4. Publicación de Resultados en el Topic de Salida.....	6
3. Resultados.....	7
3.1. Descripción de los Resultados Obtenidos.....	7
3.2. Estructura del Mensaje de Salida y Validación.....	7
4. Conclusión.....	9
4.1. Resumen del Proceso.....	9
4.2. Principales Logros.....	9

1. Introducción

1.1. Contexto

Esta segunda parte del Sprint 5 representa la continuación de la arquitectura de procesamiento streaming del proyecto TradeData. En los sprints anteriores hemos construido un pipeline progresivo y completo: desde la ingesta y almacenamiento de datos históricos de SOLUSD en capas Bronze, Silver y Gold (Sprints 1 a 3), pasando por su visualización mediante Amazon QuickSight (Sprint 4), hasta la adquisición de datos en tiempo real desde Binance y su publicación en Apache Kafka (Sprint 5.1). Con el topic *imat3a_SOL_BigDaddyks* recibiendo mensajes de cotización minuto a minuto, el Sprint 5.2 da el último paso: añadir la capa de análisis al flujo streaming, calculando el indicador VWAP mediante Spark Structured Streaming y publicando los resultados en un topic de salida.

Esta parte sitúa al proyecto en la tercera etapa del flujo de procesamiento streaming:
Data Acquisition → Data Storage → **Data Analysis** → Results.

1.2. Objetivos del Sprint 5.2

Los objetivos concretos del Sprint 5.2, tal y como se recogen en la HU-7, son los siguientes:

- Desarrollar un job de Spark Structured Streaming que consuma los mensajes de cotización de SOLBTC almacenados en el topic *imat3a_SOL_BigDaddyks*.
- Calcular el indicador VWAP (Volume Weighted Average Price) por ventanas temporales de 5 minutos aplicando la fórmula $VWAP = \frac{\sum(\text{precio} \times \text{volumen})}{\sum(\text{volumen})}$.
- Publicar los resultados en el topic de salida *imat3a_SOL_BigDaddyks_VWAP*, con la clave igual al símbolo de la criptomoneda y el valor en formato JSON incluyendo los campos *window_start*, *window_end*, *symbol* y *vwap*.

1.3. Justificación

El VWAP es el precio "pata negra": al meter el volumen en la ecuación, es mucho más fiable que el precio de cierre normal. Calcularlo en ventanas de 5 minutos nos permite ver qué está pasando de verdad en el intradía sin comernos el ruido de operaciones pequeñas.

Para el montaje técnico, usamos Spark Structured Streaming porque:

- Es el estándar para analizar datos en tiempo real (junto a Flink).
- Se lleva de lujo con Kafka.
- Como ya usamos Spark en el Sprint 3, no nos complicamos la vida y seguimos con la misma tecnología.

2. Metodología

2.1. Respuesta a HU-7: Análisis en Tiempo Real con Spark Structured Streaming

La HU-7 establece que el equipo debe implementar un sistema de análisis en tiempo real que, a partir de los mensajes de cotización almacenados en el topic de Kafka del Sprint 5.1, calcule el VWAP por ventanas de 5 minutos y publique los resultados en un topic de salida. Para abordar esta historia hemos desarrollado el script *SparkStreamingApp.py*, que implementa el pipeline completo en cuatro fases bien diferenciadas: lectura del topic de entrada, transformación y parseo del JSON, cálculo del VWAP mediante agregación por ventanas, y escritura del resultado en el topic de salida.

El pipeline toma como prerequisite que el producer del Sprint 5.1 (*solusd_real_time.py* + *kafka_simple_producer.py*) esté en ejecución y publicando mensajes en el topic *imat3a_SOL_BigDaddyks*. Spark actúa como consumidor de ese topic, procesa los datos en micro-batches y produce un flujo derivado con el VWAP calculado. El consumer actualizado del Sprint 5.1 (*kafka_simple_consumer.py*) permite verificar simultáneamente los mensajes de ambos topics, mostrando tanto las velas individuales del producer como los resultados VWAP del job de Spark.

2.2. Configuración del Job de Spark Structured Streaming

El job se inicializa creando una *SparkSession* con el nombre de aplicación *Calculo_VWAP_SOL*. La lectura del topic de entrada se configura mediante el conector Kafka de Spark, con los siguientes parámetros:

- Topic de entrada: *imat3a_SOL_BigDaddyks*
- Bootstrap server: *51.49.235.244:9092*
- Protocolo de seguridad: *SASL_PLAINTEXT* con mecanismo *PLAIN*
- Máximo de offsets por micro-batch: 5, lo que limita el volumen de mensajes procesados en cada trigger y estabiliza la latencia de procesamiento

Los mensajes llegan de Kafka en formato binario. El job los convierte a *String* mediante una expresión *selectExpr* y aplica la función *from_json* con un schema definido explícitamente, que describe la estructura del JSON publicado por el producer del Sprint 5.1:

- *symbol: StringType*
- *@timestamp: TimestampType*
- *close: StringType*
- *volume: StringType*

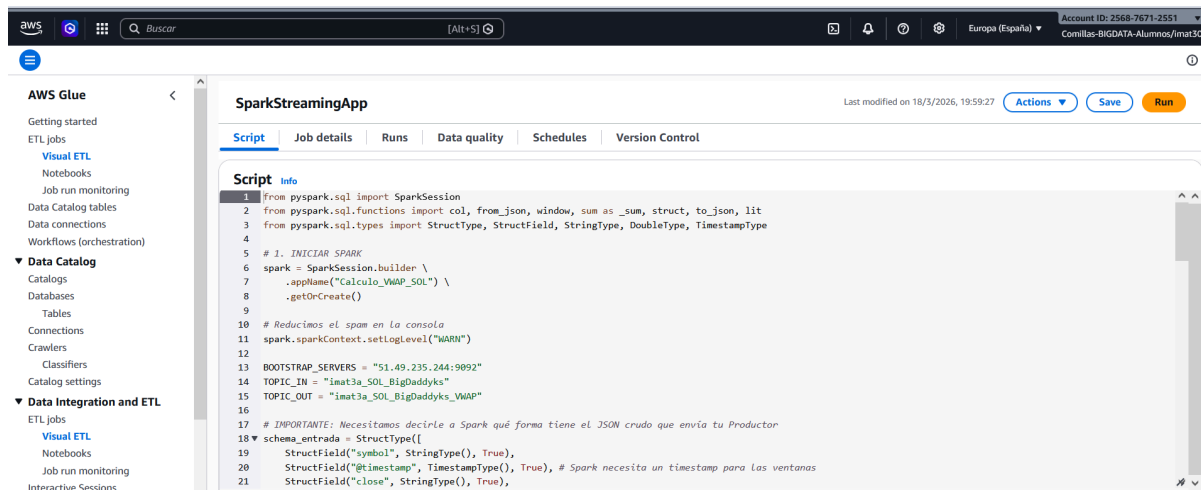


Figura 1. Job created

La decisión de definir close y volume como *StringType* en el schema responde a que el producir los publica como strings directamente a partir de los campos del websocket de Binance, que los envía en ese formato. El casteo a *Double* se realiza en el paso posterior, dentro del cálculo del VWAP.

El trigger del job se configura con un *processingTime* de 1 minuto, lo que significa que Spark lanza un nuevo micro-batch cada minuto, acumulando los mensajes recibidos en ese intervalo antes de procesarlos. El checkpoint se persiste en */tmp/spark_checkpoint_vwap_v2*, lo que permite al job recuperarse de posibles fallos sin reprocesar mensajes ya procesados.

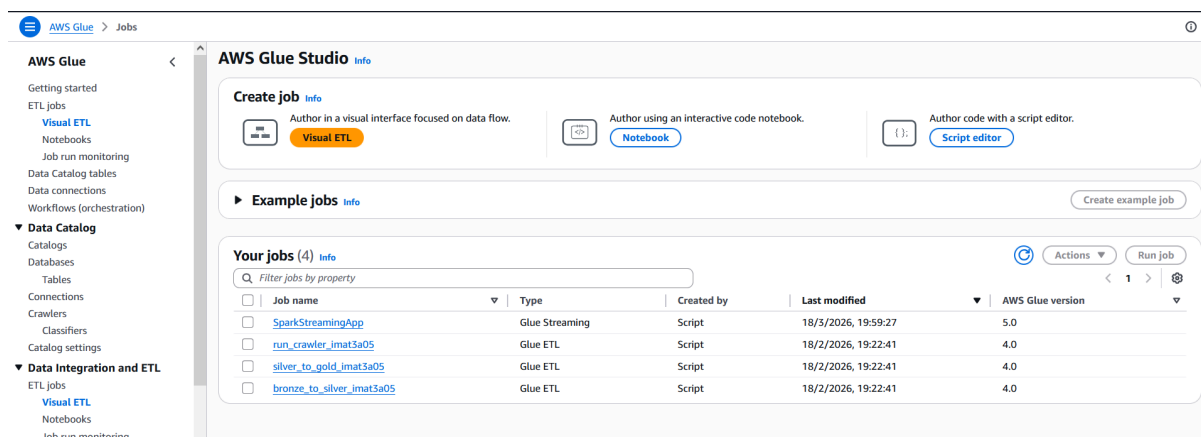


Figura 2. Job created

2.3. Cálculo del VWAP por Ventanas de 5 Minutos

El cálculo del VWAP se implementa en tres pasos sucesivos sobre el DataFrame de streaming.

En primer lugar se realiza un pre-cálculo en el que se generan dos columnas derivadas: *precio_x_volumen*, resultado de multiplicar *close* casteado a *Double* por *volume* casteado a *Double*, y *volume_num*, que es simplemente *volume* casteado a *Double*. Estas columnas intermedias son las que se agregarán en el paso siguiente.

A continuación se aplica la agrupación por ventana temporal y símbolo. Spark agrupa los eventos usando la función `window` sobre el campo `@timestamp` con una duración de 5 minutos, lo que define ventanas de tipo tumbling (no solapadas): cada evento pertenece a exactamente una ventana, y los límites son fijos en el tiempo (00:00–00:05, 00:05–00:10, etc.). Sobre cada grupo se calculan dos sumas: `sum_pv` como suma acumulada de `precio_x_volumen`, y `sum_v` como suma acumulada de `volume_num`.

Por último, sobre el DataFrame agrupado se calcula la columna `vwap` como el cociente `sum_pv / sum_v`, que corresponde exactamente a la fórmula de la HU-7:

$$VWAP = \Sigma(\text{precio} \times \text{volumen}) / \Sigma(\text{volumen}).$$

2.4. Publicación de Resultados en el Topic de Salida

El DataFrame con el VWAP calculado se transforma para ajustarse al formato de mensaje que exige la HU-7: la clave de Kafka se establece como el campo `symbol`, y el valor se serializa como un JSON construido con la función `to_json` sobre un struct que contiene los cuatro campos requeridos: `window_start`, `window_end`, `symbol` y `vwap`. Spark accede a los límites de la ventana a través de los campos `window.start` y `window.end` del objeto `window` generado por la función de agrupación.

La escritura en Kafka se configura con los siguientes parámetros:

- Topic de salida: `imat3a_SOL_BigDaddyks_VWAP`
- `outputMode`: `update`, lo que significa que Spark emite una fila de resultado cada vez que el valor de una ventana se actualiza, es decir, cada vez que llega un nuevo mensaje que pertenece a esa ventana y el micro-batch se procesa. Esto implica que una misma ventana puede producir múltiples mensajes de salida a medida que se van acumulando datos dentro de ella, con el VWAP recalculado en cada trigger.
- Checkpoint: `/tmp/spark_checkpoint_vwap_v2`

El consumer actualizado en el Sprint 5.2 se suscribe simultáneamente a ambos topics mediante el método `subscribe` (en lugar del `assign` utilizado en el Sprint 5.1), y diferencia la lógica de presentación según el topic de origen: para el topic de entrada imprime los campos estándar de la vela (key, value, offset, timestamp), mientras que para el topic de salida extrae y presenta de forma estructurada los campos `symbol`, `window_start`, `window_end` y `vwap` del JSON recibido.

Job name	Run status	Type	Start time (Local)	End time (Local)	Run time	Capacity	Worker type	DPU hours
SparkStreamingApp	Running	Glue Streaming	03/19/2026 18:34:50	-	-	2	G.1X	-
SparkStreamingApp	Canceled	Glue Streaming	03/18/2026 19:59:30	03/18/2026 20:05:45	5 minutes	2	G.1X	0.20
SparkStreamingApp	Canceled	Glue Streaming	03/18/2026 19:46:49	03/18/2026 19:59:14	11 minutes	2	G.1X	0.40
SparkStreamingApp	Canceled	Glue Streaming	03/18/2026 19:36:23	03/18/2026 19:45:50	8 minutes	2	G.1X	0.30

Figura 3. Job running

3. Resultados

3.1. Descripción de los Resultados Obtenidos

Con el producer del Sprint 5.1 en ejecución publicando velas de *SOLBTC* cada minuto, el job de Spark Structured Streaming consume los mensajes del topic *imat3a_SOL_BigDaddyks* y produce resultados VWAP en el topic *imat3a_SOL_BigDaddyks_VWAP* con una cadencia de 1 minuto por trigger. Dado que las ventanas tienen una duración de 5 minutos y el *outputMode* es *update*, cada ventana activa genera hasta 5 mensajes de salida a lo largo de su ciclo de vida: uno por cada micro-batch en el que llega al menos un mensaje perteneciente a esa ventana, con el VWAP recalculado de forma acumulativa en cada ocasión.

Producer:

```
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-18T18:58:59Z', 'close': '0.00125980', 'volume': '4.08200000'}
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-18T18:59:59Z', 'close': '0.00126000', 'volume': '697.56400000'}
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-18T19:00:59Z', 'close': '0.00126030', 'volume': '118.74200000'}
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-18T19:01:59Z', 'close': '0.00126010', 'volume': '7.72300000'}
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-18T19:02:59Z', 'close': '0.00125990', 'volume': '6.86200000'}
```

Figura 4. Producer mandando mensaje

La clave de cada mensaje de salida es el símbolo de la criptomoneda (*SOLBTC*, procedente del campo *symbol* del JSON de entrada), y el valor es un JSON con la estructura completa que exige la HU-7. Los límites de la ventana (*window_start* y *window_end*) quedan fijados desde el primer mensaje de la ventana y no varían entre actualizaciones; lo que se actualiza en cada trigger es únicamente el valor del *vwap*, que se recalcula con los nuevos datos acumulados.

La verificación de los resultados se realiza ejecutando el consumer, que muestra en tiempo real los mensajes de ambos topics de forma diferenciada. Para el topic de salida presenta el símbolo, el intervalo de la ventana y el VWAP calculado, lo que permite comprobar visualmente la coherencia del indicador con los precios y volúmenes que están llegando por el topic de entrada.

Consumer:

```
-----
key:      SOLUSD
value:    {'symbol': 'SOLBTC', '@timestamp': '2026-03-18T19:02:59Z', 'close': '0.00125990', 'volume': '6.86200000'}
offset:   31
timestamp: 1773860580216
-----
Moneda:   SOLBTC
Ventana:  2026-03-18T19:00:00.000Z a 2026-03-18T19:05:00.000Z
VWAP:     0.001260287786344048
-----
```

Figura 5. Consumer pinteando los mensajes recibidos

3.2. Estructura del Mensaje de Salida y Validación

Un mensaje publicado en el topic *imat3a_SOL_BigDaddyks_VWAP* tras el procesamiento de una ventana tiene la siguiente estructura:

Clave (key): SOLBTC

Valor (value): { "window_start", "window_end", "symbol", "vwap" }

Hemos verificado el correcto funcionamiento del pipeline comprobando que:

- El campo `symbol` del mensaje de salida coincide con el campo `symbol` del mensaje de entrada (`SOLBTC`), ya que Spark lo propaga directamente desde el JSON de entrada a través de la agrupación.
 - Los campos `window_start` y `window_end` delimitan correctamente intervalos de 5 minutos alineados con el tiempo UTC, derivados del campo `@timestamp` de los mensajes de entrada, que a su vez proviene del timestamp de cierre de la vela en Binance.
 - El valor de `vwap` es un número decimal que refleja el precio medio ponderado por volumen de los mensajes recibidos dentro de la ventana hasta el momento del trigger, y se recalcula correctamente con cada nuevo mensaje acumulado.
 - La clave del mensaje de salida (`SOLBTC`) coincide con el campo `symbol`, tal y como establece la HU-7.
-

4. Conclusión

4.1. Resumen del Proceso

En esta segunda parte del sprint hemos implementado el job de Spark Structured Streaming *SparkStreamingApp.py*, que consume los mensajes de cotización de *SOLBTC* del topic *imat3a_SOL_BigDaddyks*, calcula el VWAP por ventanas de 5 minutos aplicando la fórmula $\Sigma(\text{precio} \times \text{volumen}) / \Sigma(\text{volumen})$, y publica los resultados en el topic *imat3a_SOL_BigDaddyks_VWAP* con un trigger de 1 minuto y *outputMode update*. La clave del mensaje de salida es el símbolo de la criptomoneda y el valor es un JSON con los campos *window_start*, *window_end*, *symbol* y *vwap*, en exacta conformidad con lo que establece la HU-7.

Hemos actualizado también el consumer para que sea capaz de monitorizar simultáneamente ambos topics, diferenciando la presentación de los mensajes de velas individuales y de los resultados VWAP, lo que facilita la verificación del pipeline completo en tiempo real.

4.2. Principales Logros

Los logros principales del Sprint 5.2 incluyen:

1. Primer indicador calculado en tiempo real: Hemos implementado el cálculo del VWAP sobre datos reales de mercado en streaming, añadiendo la capa de Data Analysis a la arquitectura del proyecto y completando así el flujo completo que la teoría de la asignatura define para el procesamiento streaming.
2. Integración end-to-end del pipeline streaming: El proyecto cuenta ahora con un pipeline Binance → Kafka → Spark Structured Streaming → Kafka completamente operativo, que va desde la adquisición del dato en el websocket de Binance hasta la producción de un indicador derivado en tiempo real.
3. Uso de ventanas temporales con Spark: Hemos aplicado la función window de Spark Structured Streaming para agrupar eventos por intervalos de 5 minutos usando el event time del mensaje (el timestamp de cierre de la vela de Binance), lo que garantiza que el VWAP refleja el tiempo real del mercado y no el tiempo de procesamiento.
4. Consumer unificado para monitorización del pipeline: La actualización del consumer para suscribirse simultáneamente a los dos topics mediante subscribe proporciona una herramienta de observabilidad sencilla que permite verificar en tiempo real tanto el flujo de velas entrantes como los resultados VWAP producidos por el job de Spark.